



2003-2023

2023年第十一届“英特尔”杯全国并行应用挑战赛

循环优化

“INTEL” Cup Parallel Application Challenge (PAC 2013-2023)

主讲人：生汉森-山东大学



本期培训总体规划

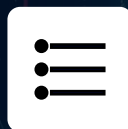
说明

1. 本届赛事培训活动由2022年并行计算相关赛事总决赛获奖队伍队长或主力成员提供;
2. 直播时间为8月4日-9月9日 (17:00-18:00 支持回放) , 具体日期请见课表;
3. 请关注「并行科技」微信公众号。「并行直播课」B站直播间或加入「PAC 2023 竞赛交流群」(微信: 18310726311) 及时获取直播预告;
4. 本届赛事培训规划如下, 如遇调整组委会将通过「微信公众号」及「微信交流群」及时告知。

序号	课程	主讲人	日期	时间
1	性能分析工具	山东大学 生汉森	8月4日 (周五)	17:00-18:00
2	并行程序的插装分析 精度量化和误差分析	中国科学院大学 邱霁岩	8月11日 (周五)	17:00-18:00
3	访存优化	华中科技大学 王绍宇	8月14日 (周一)	17:00-18:00
4	循环优化	山东大学 生汉森	8月18日 (周五)	17:00-18:00
5	更快的排序 + 循环分块	中国科学院大学 邱霁岩	8月28日 (周一)	17:00-18:00
6	软件流水线并行优化	华中科技大学 王绍宇	9月1日 (周五)	17:00-18:00
7	任务流 + __gnu_parallel	中国科学院大学 邱霁岩	9月8日 (周五)	17:00-18:00

目录

CONTENTS



01.循环预处理



02.循环交换



03.循环合并



04.循环拆分



05.循环展开



06.向量化



07.循环分块



08.循环倾斜



准备知识——循环中的数据依赖

循环中的数据依赖：循环中的数据依赖（Circular Data Dependency）是指在一个循环结构中，存在数据的相互依赖关系，即某个数据的计算或更新依赖于另一个数据，而该数据又依赖于其他数据，形成了一个依赖链条

```
1. for(size_t i=0;i<n;i++){  
2.     A[i+1]=A[i]+1;  
3. }
```

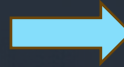
```
1. for(size_t i=0;i<n;i++){  
2.     A[i]=i;  
3.     B[i]=A[i]+1;  
4.     C[i]=A[i-1]+B[i];  
5. }
```




循环预处理

循环预处理：循环预处理是指在循环执行之前对循环中的某些计算进行提前处理，以减少在每次循环迭代中的重复计算

```
1. int sum,num;
2. for (size_t i = 0; i < N; i++){
3.     int average=sum/count;
4.     for (size_t j = 0; j < average; j++){
5.         A[i][j]=B[i]*C[i]+j;
6.     }
7. }
```



```
1. int sum,num;
2. int average=sum/count;
3. for (size_t i = 0; i < N; i++){
4.     int tmp=B[i]*C[i];
5.     for (size_t j = 0; j < average; j++){
6.         A[i][j]=tmp+j;
7.     }
8. }
```

本质：将重复操作提出循环外，减少计算量

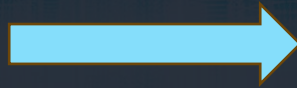
注意：必须保证正确性，保证将计算提出循环后的代码和之前是等价的



循环交换

循环交换：通过改变循环的嵌套顺序来提高程序的性能。循环交换通常用于处理多层嵌套的循环结构，通过改变循环的执行顺序，可以改善数据的访问模式、缓存利用和并行性。

```
1. int A[M][N], B[N][K], C[M][K];
2. for(size_t i=0; i<M; i++){
3.     for(size_t j=0; j<N; j++){
4.         for(size_t k=0; k<K; k++){
5.             C[i][j] += A[i][k] * B[k][j];
6.         }
7.     }
8. }
```



```
1. int A[M][N], B[N][K], C[M][K];
2. for(size_t i=0; i<M; i++){
3.     for(size_t k=0; k<K; k++){
4.         for(size_t j=0; j<N; j++){
5.             C[i][j] += A[i][k] * B[k][j];
6.         }
7.     }
8. }
```

本质：访存优化，保证顺序访存，提高数据局部性

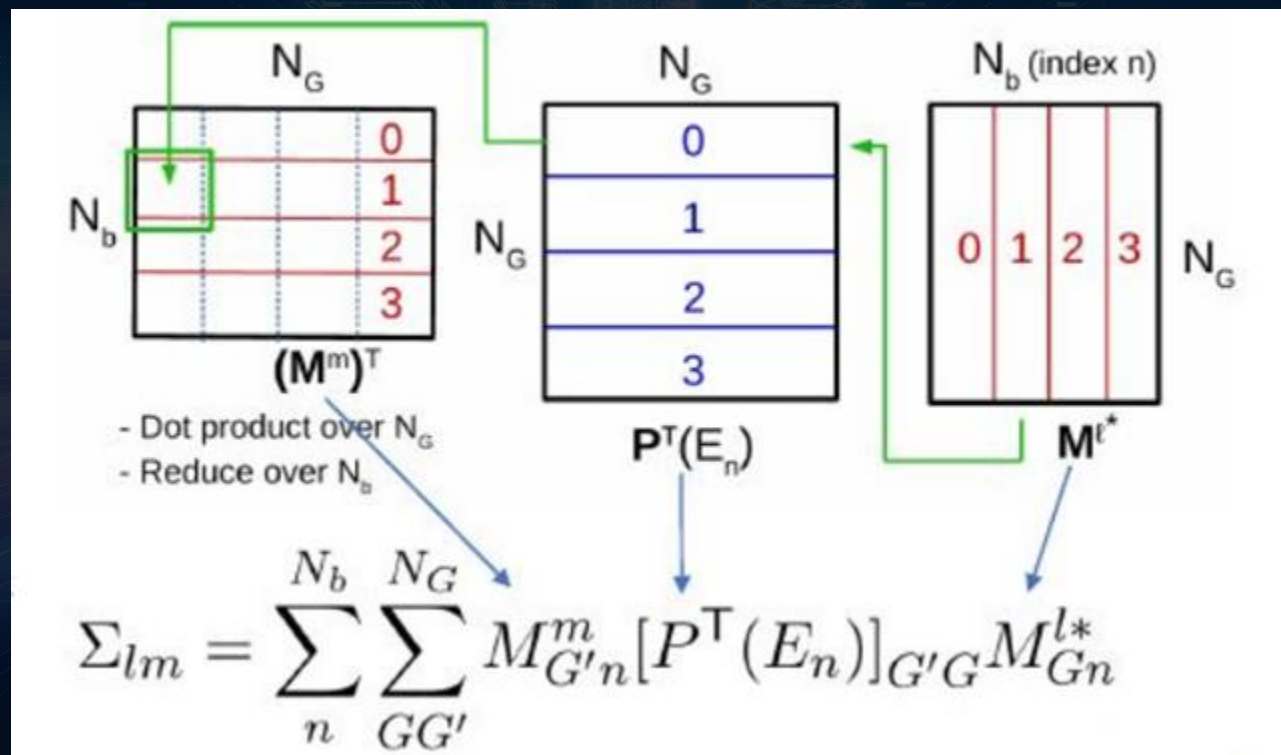
注意：

- 必须合法，不能有循环依赖。
- 循环交换之后不能产生新的不连续访问。



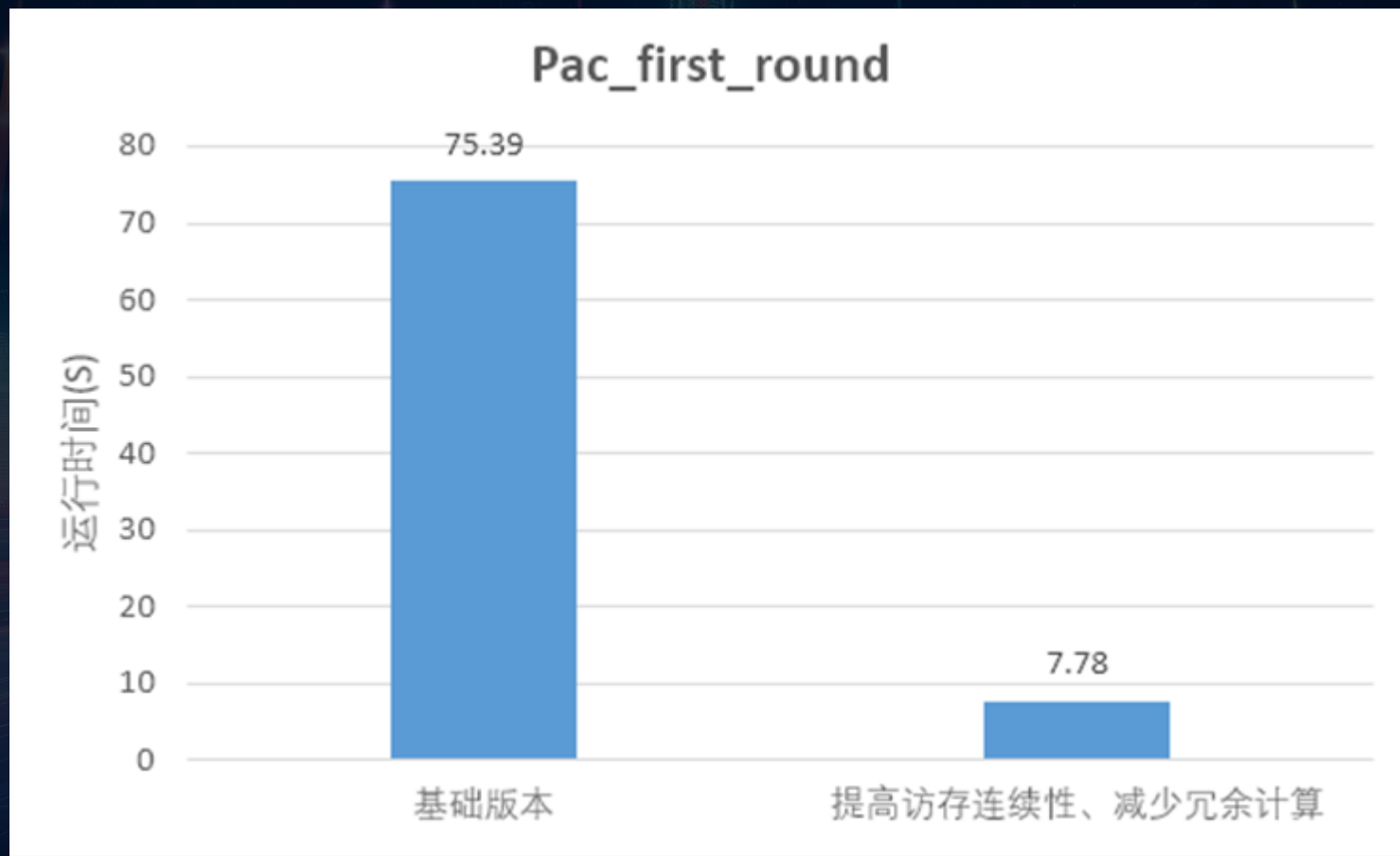
实战一：Generalized Plasmon-pole模型第一性原理计算

Generalized Plasmon-pole模型第一性原理计算GW估计的一种通用模型，应用于量子计算、能量存储与转换、光伏、纳米电子学等领域。其计算核心算法如图所示，计算复数的矩阵向量相关性，每个单元是一个类似张量计算的累加计算，算法的实现是四重循环，计算的复杂度高，且适合GPU计算。





实战一：Generalized Plasmon-pole模型第一性原理计算





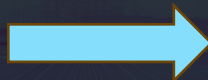
循环合并

循环合并：循环合并是指将拥有相同迭代空间的多个循环合并成一个循环。

优点：

- 有利于增强数据局部性，减少cache miss
- 减少循环迭代开销
- 多个循环合并为一个，只需要进行一次线程的启动和同步

```
1. for (size_t i = 0; i < count; i++){  
2.     C[i]=A[i]*i;  
3. }  
4. for (size_t i = 0; i < count; i++){  
5.     B[i]=A[i]*C[i];  
6. }
```



```
for (size_t i = 0; i < count; i++)  
{  
    C[i]=A[i]*i;  
    B[i]=A[i]*C[i]  
}
```

注意：必须保证正确性，不能违背原有的循环中数据依赖关系

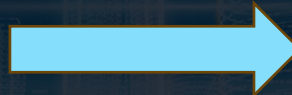


循环合并

注意：必须保证正确性，不能违背原有的循环中数据依赖关系

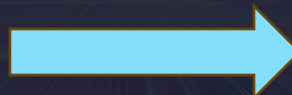
```
1. for (size_t i = 0; i < count; i++){  
2.     C[i]=A[i]*i;  
3. }  
4. for (size_t i = 0; i < count; i++){  
5.     B[i]=A[i]*C[i+1];  
6. }
```

×



```
1. for (size_t i = 0; i < count; i++){  
2. {  
3.     C[i]=A[i]*i;  
4.     B[i]=A[i]*C[i+1]  
5. }
```

✓



```
1. C[0]=A[0]*0;  
2. for (size_t i = 0; i < count; i++){  
3.     C[i+1]=A[i+1]*(i+1);  
4.     B[i]=A[i]*C[i+1];  
5. }
```

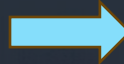
并非所有的循环合并都会取得比较好的效果，只有合并之后对数据局部性有利时才会有比较显著的效果。



循环拆分

循环拆分：循环拆分是指将一个比较复杂的循环拆分成几个小循环，原循环可能由于数据依赖的原因无法进行并行或优化，将其拆分以后即可对部分进行并行或优化。

```
1. for(size_t i=0;i<n;i++){  
2.     A[i]=i;  
3.     B[i]=A[i]+1;  
4.     C[i]=A[i-1]+B[i];  
5. }
```



```
1. for(size_t i=0;i<n;i++){  
2.     A[i]=i;  
3.     B[i]=A[i]+1;  
4. }  
5. for (size_t i = 0; i < n; i++){  
6.     C[i]=A[i-1]+B[i];  
7. }
```

注意：必须保证正确性，保证拆分前后代码是等价的



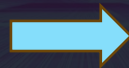
循环展开

循环展开：循环展开的基本思想是将循环体内的代码复制多次，使每次循环迭代处理多个元素，从而减少循环迭代的次数。

优点：

1. 降低循环的迭代次数判断和循环开销
2. 有利于指令流水线的调度，可以获得更好的指令级并行
3. 增加了寄存器的重用

```
1. for (size_t i = 0; i < count; i++){  
2.     A[i] += C[i] * B[i];  
3. }
```



```
1. size_t i = 0;  
2. for (i; i < count; i+=4){  
3.     A[i] += C[i] * B[i];  
4.     A[i+1] += C[i+1] * B[i+1];  
5.     A[i+2] += C[i+2] * B[i+2];  
6.     A[i+3] += C[i+3] * B[i+3];  
7. }  
8. for (i; i < count; i++){  
9.     A[i] += C[i] * B[i];  
10. }
```

注意：

保证展开后程序等价，不能有循环依赖
注意余数部分的处理

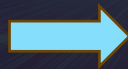


向量化

向量化:利用处理器的SIMD (Single Instruction, Multiple Data) 指令集, 将多个独立的数据操作合并为一条指令, 从而并行地处理多个数据元素, 提高程序的性能。

循环展开是向量化的基础

```
1. size_t i = 0;
2. for (i; i < count; i+=4){
3.     A[i]+=C[i]*B[i];
4.     A[i+1]+=C[i+1]*B[i+1];
5.     A[i+2]+=C[i+2]*B[i+2];
6.     A[i+3]+=C[i+3]*B[i+3];
7. }
8. for (i; i < count; i++){
9.     A[i]+=C[i]*B[i];
10. }
```



```
size_t i = 0;
for (i; i < count; i+=4){
    __m256d A_4 = _mm256_load_pd(A+i);
    __m256d B_4 = _mm256_load_pd(B+i);
    __m256d C_4 = _mm256_load_pd(C+i);
    __m256d tmp = _mm256_mul_pd(C_4,B_4);
    A_4 = _mm256_add_pd(A_4,tmp);
    _mm256_store_pd(A+i,A_4);
}
for (i; i < count; i++){
    A[i]+=C[i]*B[i];
}
```




实战二——度量空间中支撑点计算

假设要处理的数据集是 $S = \{x_i \mid i = 1, 2, \dots, n\}$, 共有 n 个数据点, 任两点间的距离可以由距离函数 $d(\cdot, \cdot)$ 计算; 要选择 k 个点作为支撑点, 标记为 $P = \{p_j \mid j = 1, 2, \dots, k\}$ 。对于 S 中任意的一个数据点 x , 其基于支撑点集合重建的坐标是其到各支撑点的距离形成的向量,

$$x^P = (d(x, p_1), \dots, d(x, p_k))$$

本项目采用距离和目标函数, 即任意两点的重建坐标间的切比雪夫距离的和, 越大越好:

$$\sum L_\infty(x^P, y^P), x, y \in S$$

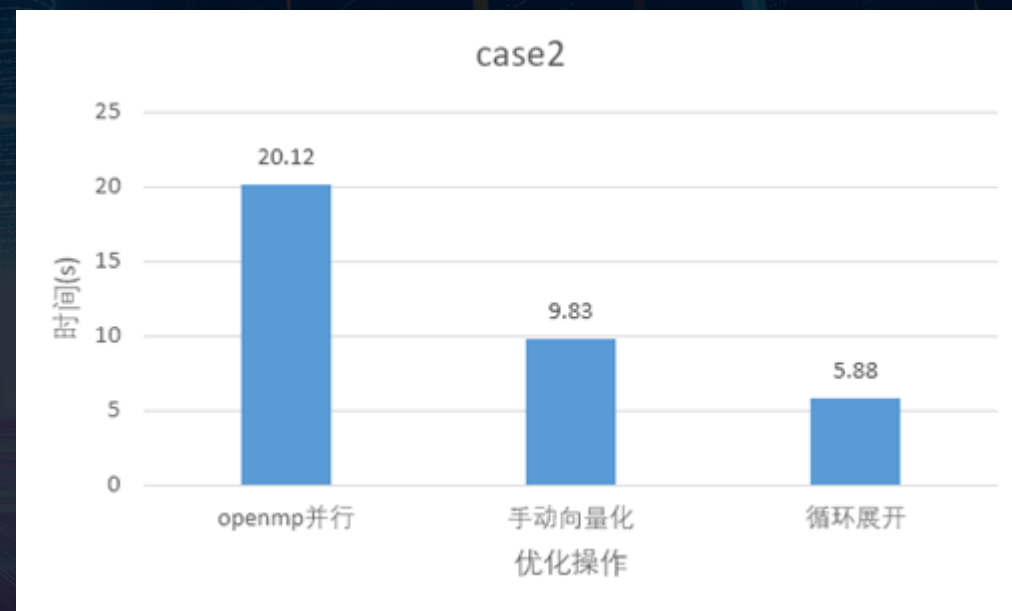
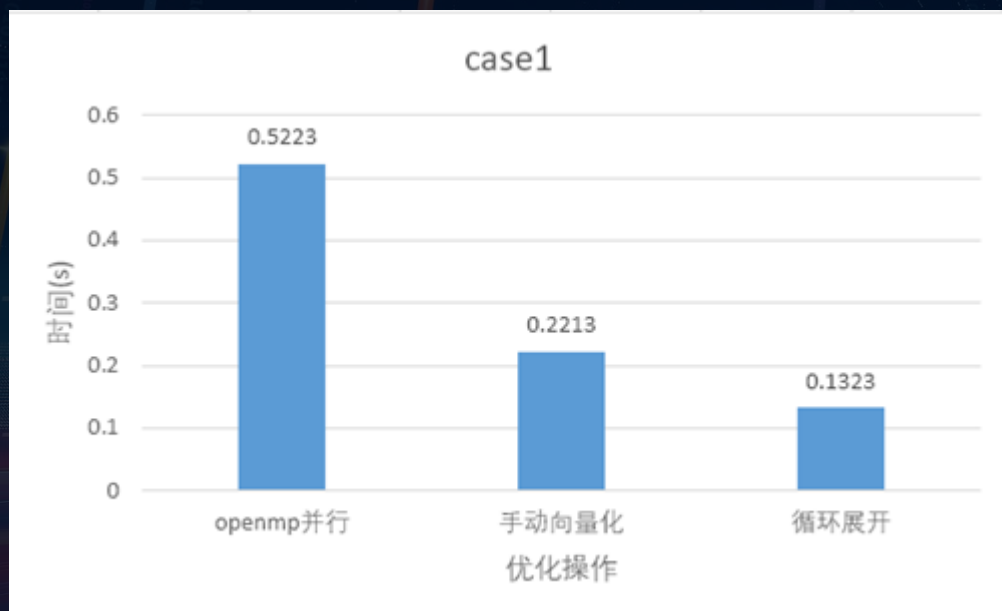
$$\text{其中 } L_\infty((a_1, a_2, \dots, a_k), (b_1, b_2, \dots, b_k)) = \max_i (|a_i - b_i|)$$

需要准确地求出 (可采用但不限于穷举法, 但要保证结果的正确性) 目标函数值最大和最小的各 1000 个支撑点集合。

```
Combination(ki, ...) {  
  if (ki == k-1):  
    for (i: pivots[ki-1]+1 .. n): // 枚举最后一位  
      pivots[k-1] = i  
  
    chebSum = getChebSum(pivots)  
  
    sort and get top 1000 max/min  
  
  else:  
    for (i: pivots[ki-1]+1 .. n)  
      Combination(ki+1)  
}
```




实战二——度量空间中支撑点计算





循环分块

循环分块(loop tiling): 它通过将大循环分割成多个小块, 以提高数据访问的局部性从而提高缓存的命中率, 最终提高程序的性能。

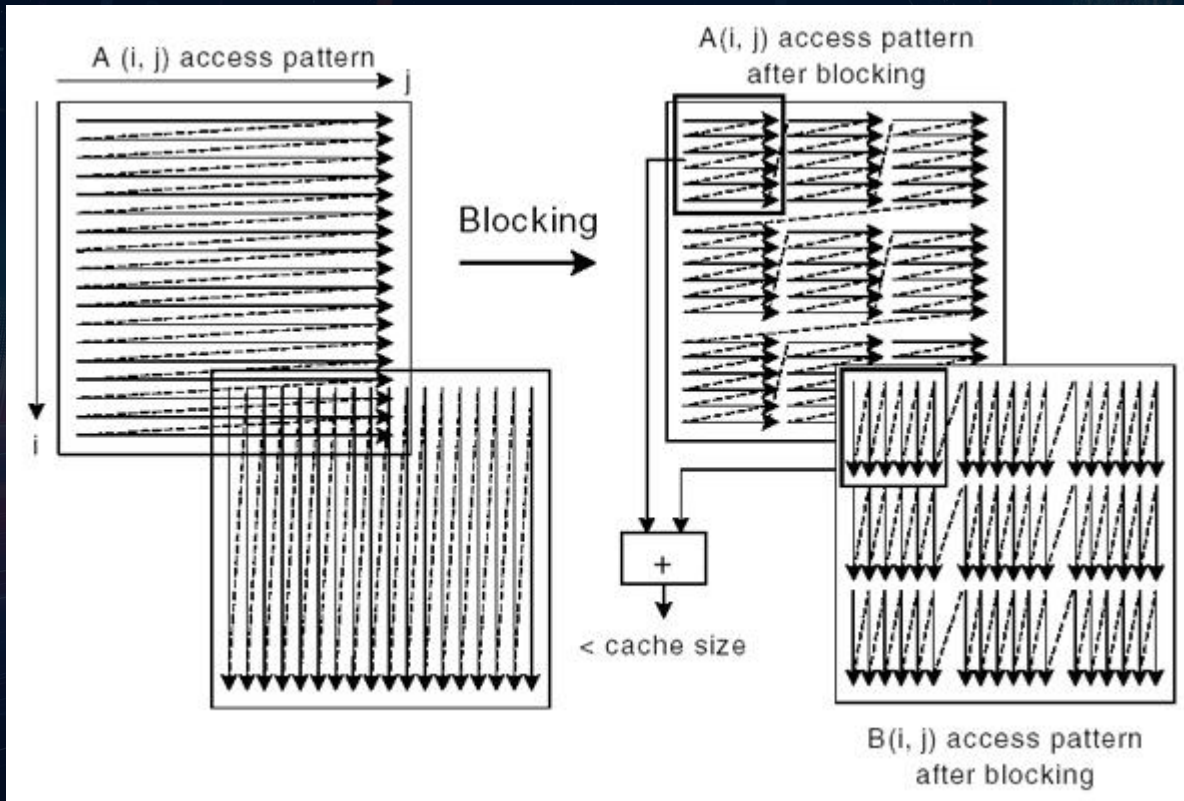
```
for (size_t i = 0; i < M; i++){  
    for (size_t j = 0; j < N; j++){  
        A[i]+=B[j];  
    }  
}
```

```
#define blk_size 4  
for (size_t i = 0; i < M; i+=blk_size){  
    for (size_t j = 0; j < N; j++){  
        for (size_t ii = i; ii < min(i+blk_size-1,M); ii++){  
            A[ii]+=B[j];  
        }  
    }  
}
```




循环分块

循环分块(loop tiling): 它通过将大循环分割成多个小块, 利用数据访问的局部性提高缓存的命中率, 从而提高程序的性能。



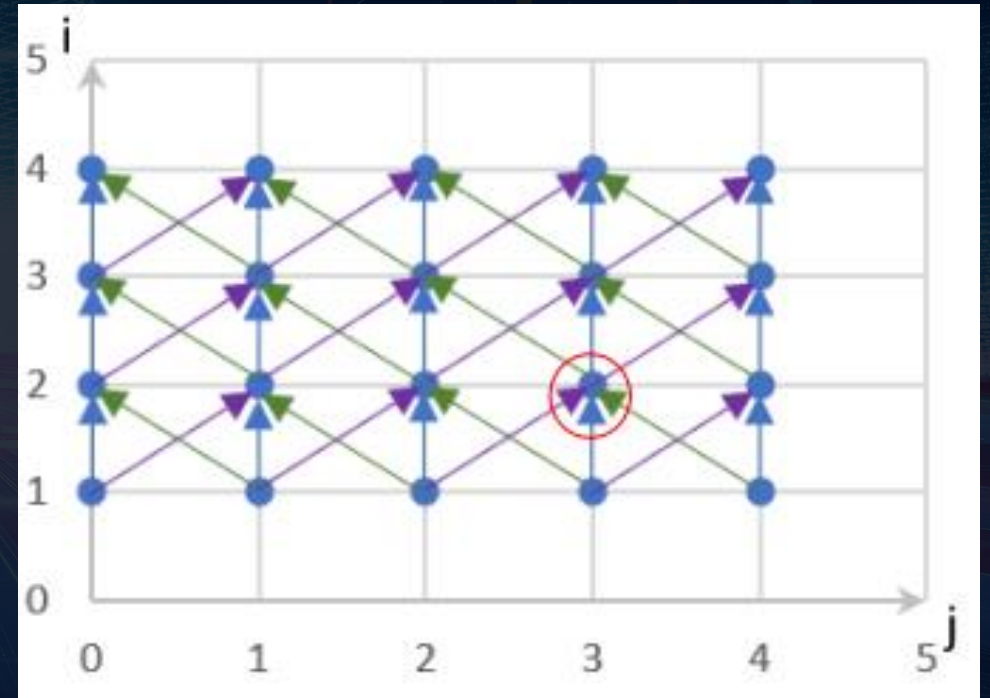
```
DO J = 1, N, Tj
  DO K = 1, N, T
    DO I = 1, N, S
      DO jj = J, min(N, J+Tj-1)
        DO kk = K, min(N, K+T-1)
          DO ii = I, min(N, I+S-1)
            C(ii, jj) = C(ii, jj) + A(ii, kk) * B(kk, jj)
```




循环倾斜

循环倾斜:循环倾斜通过改变循环索引的计算方式, 消除循环中的数据依赖, 使原有的循环变得可并行。

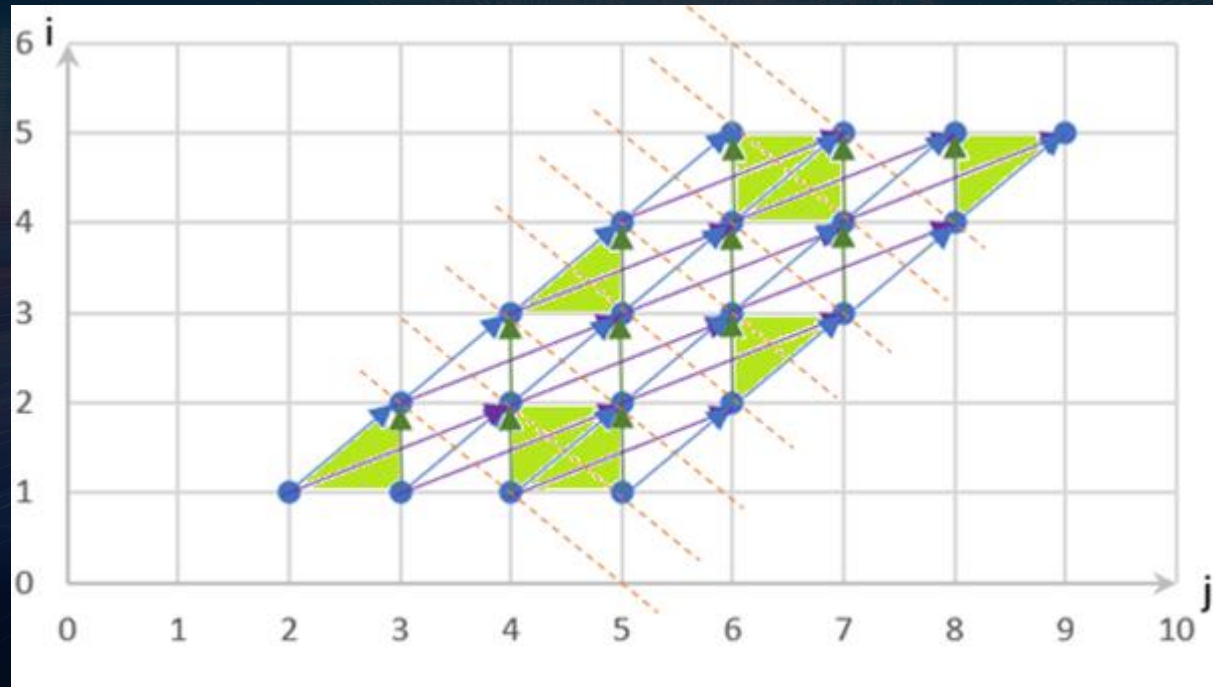
```
for (int i = 0; i < N; i++){  
    for (int j = 1; j < N; j++){  
        a[i+1][j] = a[i][j+1] -  
                    a[i][j] + a[i][j-1];  
    }  
}
```





循环倾斜

```
for (int i_new = 0; i_new < N; i_new++){  
    for (int j_new = i+1; j_new < i+N; j_new++){  
        a[i_new+1][j_new-i_new] = a[i_new][j_new-i_new+1] -  
            a[i_new][j_new-i_new] + a[i_new][j_new-i_new-1]  
    }  
}
```





循环倾斜

多面体模型编译技术将代码的多层循环抽象为多维空间，将每个计算实例抽象为空间中的点，实例间的依赖关系抽象为空间中的线，主要对循环进行优化。该算法的主要思想是针对输入代码的访存特点进行建模，调整循环语句中的每一个实例的执行顺序，使得新调度下的循环代码有更好的局部性和并行性。

给大家提供一种思路，由于数据依赖无法并行时可以考虑循环倾斜来寻找解决办法

[深度学习编译之多面体模型编译——以优化简单的两层循环代码为例 - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/35444444/answer/104444444)



总结

循环优化时追求的目标：**局部性 并行性**

局部性：循环交换 循环合并 循环展开 循环分块

并行性：循环拆分 循环倾斜

循环预处理：减少计算量

向量化：使用向量硬件加速



优化作业

```
for (int n1 = 0; n1 < number_bands; ++n1) {  
    for (int my_igp = 0; my_igp < ngpown; ++my_igp) {  
        int indigp = inv_igp_index(my_igp);  
        int igp = indinv(indigp);
```

n1层循环在最外层

```
ComplexType sch_store1 =  
    ComplexType_conj(aqsmtemp(n1, igp)) * aqsntemp(n1, igp) * 0.5 *  
    vcoul(igp) * wtilde_array(my_igp, igp);
```

只有这个操作涉及到n1层循环

```
for (int ig = 0; ig < ncouls; ++ig) {  
    DataType achtemp_re_loc[nend - nstart], achtemp_im_loc[nend - nstart];  
    for (int iw = nstart; iw < nend; ++iw) {  
        achtemp_re_loc[iw] = 0.00;  
        achtemp_im_loc[iw] = 0.00;  
    }
```

```
ComplexType I_eps_sch= I_eps_array(my_igp, ig) * sch_store1;
```

```
for (int iw = nstart; iw < nend; ++iw) {  
    ComplexType wdifff =  
        wx_array(iw) - wtilde_array(my_igp, ig);  
    ComplexType delw =  
        ComplexType_conj(wdifff) *  
        (1 / (wdifff * ComplexType_conj(wdifff)).real());  
    ComplexType sch_array =  
        delw * I_eps_sch;
```

```
achtemp_re_loc[iw] += (sch_array).real();  
achtemp_im_loc[iw] += (sch_array).imag();  
}
```




下期预告

**11TH 2023第十一届“英特尔”杯
全国并行应用挑战赛**
THE 11TH "INTEL CUP" PARALLEL APPLICATION CHALLENGE (PAC-2023)

PAC大讲堂 · 并行计算进阶培训课程
第五讲 · 更快的排序+循环分块

课程提纲:

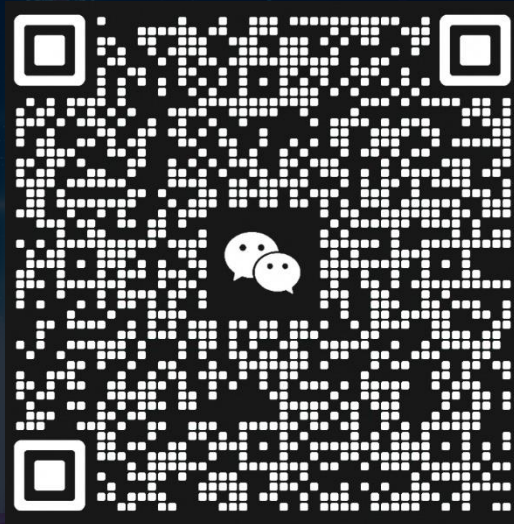
- 排序:
 - 常见排序算法回顾/key排序
 - Pairs排序/并行排序
- 循环分块
 - chcie的作用/循环分块的作用
 - 最佳分块值/举例: 矩阵向量乘
 - 举例: 矩阵矩阵乘

直播时间:
8月28日(周一) 17:00-18:00

中国科学院大学 邵野岩

「并行直播课」视频号

「并行直播课」B站直播间



- | 扫(左)码加入“PAC微信群”，技术交流答疑。
- | 扫(右)码关注“PAC公众号”，实时获取赛事动态。
- | PAC 2023 报名渠道，百度搜索关键词“并行教育”
- | 直播间：腾讯视频号 B站 搜索「并行直播课」



并行计算学习书单

基础书单

- √ 《并行计算导论》，机械工业出版社
--介绍了并行计算的主要问题、主要技术、分析方法等等。
- √ 《并行计算导论》，张林波、迟学斌、莫则尧、李若，清华大学出版社
--有若干并行计算典型算例的介绍。
- √ 《深入理解计算机系统》，机械工业出版社
--必读书目，基础中的基础。
- √ 《计算机组成与设计：硬件/软件接口》，机械工业出版社
--除了计算机体系结构，还需要尤其关注分布式系统、GPU架构设计的部分
- √ 《现代操作系统》，机械工业出版社
--对于理解并行程序行为非常重要。

进阶书单

- √ 《OpenMP核心技术指南》，机械工业出版社
- √ 《并行计算机组成与设计》，机械工业出版社
- √ 《高性能并行珠玑：多核和众核编程方法》，机械工业出版社
- √ 《并行计算的编程模型》，机械工业出版社
- √ 《高性能计算：现代系统与应用实践》，机械工业出版社
- √ 《编译原理》，机械工业出版社
- √ 《并行计算机体系结构》高等教育出版社（2002版）
- √ 《并行计算机系统结构与可扩展计算》清华大学出版社（2009版）
- √ 《智能计算系统》机械工业出版社，（2019版）
- √ 《编译原理Principles of Compiler Design》机械工业出版社
--就是传说中的龙书。尤其看代码优化的部分，理解编译器在程序优化中发挥的作用。



问题答疑及直播反馈



2003-2023



2003-2023

2023年第十一届“英特尔”杯全国并行应用挑战赛

计算永无止境

“INTEL” Cup Parallel Application Challenge (PAC 2013-2022)

TO BE CONTINUED